# HTML & CSS

Phase 3 — Expert

**Specificity · CSS Variables · Animations · BEM · @layer · Performance · Container Queries**

| | | |
|---|---|---|
| *16. CSS Specificity & Cascade* | *17. CSS Custom Properties* | *18. Animations & Transitions* |
| *19. CSS Architecture (BEM + @layer)* | *20. CSS Performance* | *21. Container Queries* |

PHASE 3

# Expert

Specificity · Variables · Animations · Architecture · Performance · Container Queries

## 16 CSS Specificity & The Cascade — Deeply Understood

When two CSS rules target the same element and same property — **who wins?** The answer is specificity. Every rule gets a 4-column score: **(inline, IDs, classes/attrs/pseudos, tags)**. Compared column by column. Higher score wins. Equal score — later rule wins.

> ■ GOTCHA
> Columns NEVER carry over. 0,0,10,0 (10 classes) LOSES to 0,1,0,0 (1 ID). No matter how many classes you stack — one ID always beats them all. Most developers don't know this.

```
/* ■■■ SPECIFICITY SCORES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
p /* 0,0,0,1 — tag */
p.intro /* 0,0,1,1 — tag + class */
.card .title /* 0,0,2,0 — two classes */
#hero p /* 0,1,0,1 — ID + tag */
#hero p.intro /* 0,1,1,1 — ID + tag + class */
style='color:red' /* 1,0,0,0 — inline — beats all above */
color: red !important /* Nuclear — beats everything */

/* Pseudo-classes count as classes (0,0,1,0) */
a:hover /* 0,0,1,1 */
.btn:focus /* 0,0,2,0 */

/* ■■■ :is() — inherits HIGHEST specificity from args ■■■ */
:is(#hero, .card) p { color: navy; }
/* Score: 0,1,0,1 — because of #hero inside :is() */

/* ■■■ :where() — ALWAYS zero specificity ■■■■■■■■■■■■■■■■■■ */
:where(#hero, .card) p { color: navy; }
/* Score: 0,0,0,1 — :where() contributes NOTHING */
/* Perfect for resets: nothing fights :where() */
:where(*, *::before, *::after) { box-sizing: border-box; }

/* ■■■ @layer — cascade control, no specificity needed ■■■ */
@layer reset, base, components, utilities;
/* Later layer WINS regardless of specificity score */

@layer components { .hero h1 { color: purple; } } /* 0,0,1,1 */
@layer utilities { .text-red { color: red; } } /* 0,0,1,0 */
/* .text-red WINS even though it has LOWER specificity */
/* Because 'utilities' layer comes AFTER 'components' */
```

| Selector type | Score | Beats? |
|---|---|---|
| * universal | 0,0,0,0 | Nothing — lowest possible |
| tag (p, h1, div) | 0,0,0,1 | Universal selector |
| class, attribute, :pseudo-class | 0,0,1,0 | Tags — regardless of count |

| | | |
|---|---|---|
| ID (#hero) | 0,1,0,0 | Any number of classes — always |
| inline style | 1,0,0,0 | IDs |
| !important | Override all | Everything — avoid in production |
| :where() | 0,0,0,0 | Nothing — use for resets |
| @layer (later) | Layer position | All earlier layers |

**Q: Rule A has one ID. Rule B has 20 class selectors. Which wins?**

A: Rule A (the ID) wins. Specificity columns never carry over — one ID (0,1,0,0) always beats any number of classes (0,0,20,0). Scores are compared column by column and they never overflow into each other. The solution to avoiding specificity wars: use BEM (single classes only), :where() for zero-specificity resets, and @layer for predictable cascade control.

**Q: What is @layer and what problem does it solve?**

A: @layer gives you explicit cascade control through a declared layer order — later layers always win over earlier layers regardless of specificity. It solves the problem of third-party library styles fighting your application styles. Put the library in an early layer, your overrides in a later layer — your styles always win without !important. Also, styles NOT in any layer beat ALL layered styles automatically.

## 17 CSS Custom Properties — Design Token System

CSS Custom Properties are not just shortcuts — they are **live, cascading, inheritable values that exist at runtime**. JavaScript can read and write them, they respond to media queries, and they can be scoped to DOM subtrees. They are the foundation of dynamic theming, dark mode, and scalable design systems.

> ■ **REAL WORLD**
> Real World: Dark mode in Next.js. Instead of duplicating all CSS with different colors, define everything as CSS variables and only change the token values under [data-theme='dark']. Every component automatically updates — zero extra CSS.

```
/* ■■■ DEFINE TOKENS ON :root ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
:root {
--color-primary: #6366f1;
--color-bg: #ffffff;
--color-surface: #f8fafc;
--color-text: #0f172a;
--color-border: #e2e8f0;
--radius-md: 8px;
--space-4: 1rem;
--shadow-sm: 0 1px 3px rgb(0 0 0 / 0.1);
--font-size-fluid: clamp(1rem, 2.5vw, 1.25rem);
}

/* ■■■ DARK MODE: only token values change ■■■■■■■■■■■■■■■■■■■■ */
[data-theme='dark'] {
--color-bg: #0f172a;
--color-surface: #1e293b;
--color-text: #f1f5f9;
--color-border: #334155;
/* --color-primary stays the same — brand is consistent */
}
```

```
/* ■■■ COMPONENTS use tokens — never raw values ■■■■■■■■■■■■ */
.card {
background: var(--color-surface);
border: 1px solid var(--color-border);
border-radius: var(--radius-md);
padding: var(--space-4);
box-shadow: var(--shadow-sm);
}

/* ■■■ SCOPED OVERRIDE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
.card--compact { --space-4: 0.5rem; } /* Only affects this subtree */

/* ■■■ FALLBACK VALUE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
color: var(--color-accent, #6366f1); /* Use fallback if undefined */

/* ■■■ JAVASCRIPT INTEGRATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
// Toggle dark mode
document.documentElement.dataset.theme = 'dark';

// Read a CSS variable
const color = getComputedStyle(document.documentElement)
.getPropertyValue('--color-primary').trim();

// Set dynamically (e.g. user-chosen accent color)
document.documentElement.style.setProperty('--color-primary', '#ef4444');
```

| Feature | CSS Custom Properties | Sass Variables |
|---|---|---|
| Lives in | Browser — at runtime | Build output — compiled away |
| JS can read/write? | Yes — getPropertyValue / setProperty | No — doesn't exist at runtime |
| Cascade & inherit? | Yes — scoped to DOM subtree | No — file level only |
| Responds to @media? | Yes — redefine inside media query | No — compile-time only |
| DevTools visible? | Yes — inspectable live | No — compiled away |
| Dark mode support? | Perfect — change tokens only | Requires duplicated CSS |

**Q: How would you implement dark mode in Next.js using CSS Custom Properties?**

A: Define all colors as CSS custom properties on :root. Create [data-theme='dark'] that overrides only the color tokens. In Next.js, create a ThemeProvider that reads from localStorage on mount and sets document.documentElement.dataset.theme. Add prefers-color-scheme as the default media query so the correct theme loads before JS runs, preventing a flash of wrong theme. Components reference only var(--color-*) tokens and automatically respond to both system preference and user toggle.

**Q: What is the difference between defining a variable on :root vs a component class?**

A: Defining on :root makes the variable globally available — the entire page inherits it. Defining on a component class scopes it to that element and its descendants only. This scoping is powerful: .card--compact { --space-4: 0.5rem } overrides spacing only for compact cards. The same component CSS works for both because var(--space-4) always resolves to the closest ancestor's definition of that variable.

## 18 CSS Animations & Transitions — Performance-First

**Transitions** animate between two states triggered by a change (hover, class toggle). **Animations** run automatically, can loop, and define multiple keyframe steps. Golden rule: **only animate transform and opacity** — GPU composited, zero layout/paint cost, silky 60fps on any device.

> ■ **GOTCHA**
> Animating width, height, top, left, or margin triggers layout recalculation on EVERY frame — causes jank and dropped frames. Use transform: scale() instead of width/height. Use transform: translate() instead of top/left. Always.

```
/* ■■■ TRANSITIONS ■■■
```

```
@media (prefers-reduced-motion: reduce) {
*, *::before, *::after {
animation-duration: 0.01ms !important;
animation-iteration-count: 1 !important;
transition-duration: 0.01ms !important;
}
}
```

| Property animated | Reflow? | Repaint? | GPU? | Verdict |
|---|---|---|---|---|
| transform (translate, scale, rotate) | No | No | Yes | Always use — cheapest |
| opacity | No | No | Yes | Always use — cheapest |
| color / background-color | No | Yes | No | Acceptable for state changes |
| box-shadow / border | No | Yes | No | Use sparingly in animations |
| width / height | Yes | Yes | No | Avoid — use transform: scale() |
| top / left / margin | Yes | Yes | No | Avoid — use transform: translate() |

**Q: What is the difference between transition and animation?**

A: Transitions animate between exactly two states triggered by a state change (hover, focus, class toggle). They don't run automatically — they need a trigger. Animations (@keyframes) run automatically when applied, can define multiple steps, loop indefinitely, run forwards and backwards, and be paused with animation-play-state. Use transitions for interaction feedback. Use animations for autonomous motion like loaders, entrance effects, or anything on page load.

**Q: Why should you never animate width, height, top, or left?**

A: These properties trigger reflow — the browser recalculates geometry of ALL affected elements on every frame. This is O(n) work per frame and causes dropped frames and jank. Use transform: scale() instead of width/height, and transform: translate() instead of top/left. Transform and opacity are GPU-composited — they skip reflow and repaint entirely, running at 60fps even on low-end mobile devices.

## 19 CSS Architecture — BEM & @layer

As projects grow, CSS becomes unmaintainable without a system. **BEM** is a naming convention that eliminates specificity wars by making every selector a single class. **@layer** is a cascade tool that gives explicit, predictable priority ordering to your stylesheets.

```
/* ■■■ BEM: Block__Element--Modifier ■■■■■■■■■■■■■■■■■■■■■■■■ */
/* Block = standalone reusable component */
/* Element = part of the block (double underscore __) */
/* Modifier = variation of block or element (double dash --) */
/* KEY BENEFIT: all selectors are single class = 0,0,1,0 */

.card { } /* Block */
.card__image { } /* Element */
.card__body { } /* Element */
.card__title { } /* Element */
.card__footer { } /* Element */
.card--featured { } /* Block modifier */
.card--dark { } /* Block modifier */
.card__title--large { } /* Element modifier */
.card__button--primary { } /* Element modifier */
```

```
/* ■■■ @layer: Explicit Cascade Priority ■■■■■■■■■■■■■■■■■■■■ */

/* Step 1: Declare all layers at the top — order = priority */
@layer reset, tokens, base, layout, components, utilities;
/* utilities is last — always wins */

@layer reset {
:where(*, *::before, *::after) { box-sizing: border-box; margin: 0; }
}
@layer tokens {
:root { --color-primary: #6366f1; --space-4: 1rem; }
}
@layer base {
body { font-family: system-ui, sans-serif; }
h1 { font-size: 2rem; }
}
@layer components {
.btn { padding: 0.5rem 1rem; border-radius: 6px; }
.card { background: var(--color-surface); }
}
@layer utilities {
.hidden { display: none; } /* Always wins over component styles */
.text-center { text-align: center; }
.mt-4 { margin-top: var(--space-4); }
}

/* Third-party library — wrap in layer so YOUR layers beat it */
@layer mui { /* import MUI styles here — now your utilities win */ }
```

| | BEM | Utility-first (Tailwind) | CSS Modules |
|---|---|---|---|
| Approach | Semantic names with convention | Single-purpose utilities | Scoped to component file |
| Specificity | All equal (0,0,1,0) | All equal (0,0,1,0) | Auto-scoped, no conflicts |
| Best for | Large teams, design systems | Rapid UI, prototyping | React/Next.js components |
| Risk of conflicts | Low with discipline | Very low | None — isolated |

**Q: What is BEM and why does it prevent specificity issues?**

A: BEM (Block Element Modifier) is a CSS naming convention where every class follows .block, .block__element, or .block--modifier. The key benefit: every selector is a single class with identical specificity of (0,0,1,0). No nesting, no IDs, no tag qualifiers. No specificity wars — the rule that appears later always wins predictably. You always know exactly which rule applies.

**Q: What happens to styles that are NOT in any @layer?**

A: Unlayered styles have higher priority than ALL layered styles — they win regardless of layer order or specificity. This is important with third-party libraries: if you import a library without wrapping it in a layer, it beats all your layered styles. The fix: @layer library { import styles here } — now your application layers win.

## 20 CSS Performance — Expert-Level Optimisation

Expert CSS means understanding how browsers render pages and writing CSS that minimises expensive operations. The browser rendering pipeline: **Style** → **Layout** → **Paint** → **Composite**. The later in the pipeline a property change happens, the cheaper it is.

```
/* ■■■ RENDERING PIPELINE COSTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
/* Cheapest: composite only (GPU handles it) */
transform, opacity

/* Medium: paint (pixels redrawn, no geometry change) */
color, background-color, box-shadow, border-color

/* Expensive: layout + paint (geometry recalculated) */
width, height, padding, margin, top, left, font-size, display

/* ■■■ will-change: pre-promote to GPU layer ■■■■■■■■■■■■■■■■ */
.card-animated { will-change: transform; }
/* WARNING: Use sparingly. Each creates a GPU layer. */
/* Too many = more memory, worse performance. Remove after: */
el.addEventListener('animationend', () => {
el.style.willChange = 'auto'; /* Remove promotion */
});

/* ■■■ contain: limit layout/paint scope ■■■■■■■■■■■■■■■■■■■■ */
.widget {
contain: layout style paint;
/* Browser only recalculates layout WITHIN this element */
/* Changes inside do NOT affect the rest of the page */
}

/* ■■■ Selector efficiency ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
/* Browser reads selectors RIGHT TO LEFT */
.nav ul li a { } /* Finds ALL <a> tags, then filters up — slow */
.nav-link { } /* Direct class lookup — O(1) — always prefer */

/* ■■■ font-display for fast text rendering ■■■■■■■■■■■■■■■■■ */
@font-face {
font-family: 'Inter';
src: url('/fonts/Inter.woff2') format('woff2');
font-display: swap;
/* Show fallback font immediately. */
/* Swap to custom font when loaded. */
/* Prevents FOIT: Flash of Invisible Text */
}

/* ■■■ Logical Properties: i18n-ready CSS ■■■■■■■■■■■■■■■■■■■ */
/* Instead of: Use: */
margin-left: 1rem; /* margin-inline-start: 1rem */
margin-right: 1rem; /* margin-inline-end: 1rem */
padding-top: 1rem; /* padding-block-start: 1rem */
width: 100%; /* inline-size: 100% */
/* Logical props work for RTL languages automatically */
```

| CSS change | Pipeline step | Cost | Better alternative |
|---|---|---|---|

| | | | |
|---|---|---|---|
| transform, opacity | Composite only | Cheapest — GPU | Already optimal |
| color, background | Paint | Medium | Acceptable for state changes |
| box-shadow, border | Paint | Medium | Avoid in loops/animations |
| width, height | Layout + Paint | Expensive | transform: scale() |
| top, left, margin | Layout + Paint | Expensive | transform: translate() |
| display, font-size | Layout + Paint | Very expensive | Avoid in animations |

**Q: What is the difference between reflow and repaint?**

A: Reflow (layout) is the browser recalculating geometry — position, size, relationships. Triggered by width, height, margin, padding, font-size, display changes. Expensive because it cascades — one change can trigger recalculation of many elements. Repaint is redrawing pixels without geometry change — triggered by color, background. Cheaper but still involves GPU work. transform and opacity bypass both through GPU compositing — always prefer them for animations.

**Q: What is font-display: swap and why does it matter?**

A: By default, browsers show invisible text (FOIT — Flash of Invisible Text) while a custom font loads. font-display: swap tells the browser to immediately show text using a fallback system font, then swap to the custom font when it finishes loading. This prevents invisible text — users can read content immediately. The trade-off is a brief visual shift when the font swaps. font-display: optional avoids the swap by only using the custom font if it loads within a very short time window.

## 21 Container Queries — Component-Level Responsive Design

Media queries respond to the **viewport size**. But a card used in a narrow sidebar and a wide main content area needs different layouts — and media queries cannot tell you the component's available space. **Container Queries respond to the parent container's size**, enabling truly self-contained, reusable components.

> ■ REAL WORLD
> Real World: A card component used in a 3-column grid (each column ~280px) AND in a hero section (~900px wide). With container queries, the same component shows stacked layout in the grid and horizontal layout in the hero — automatically. Zero duplicate CSS. Zero JavaScript.

```
/* ■■■ STEP 1: Define a containment context on the PARENT ■■ */
.card-wrapper {
container-type: inline-size; /* Track this element's width */
/* inline-size = track width only (most common) */
/* size = track both width and height */
}

/* Optional: name the container */
.sidebar {
container-type: inline-size;
container-name: sidebar;
}

/* ■■■ STEP 2: Write container queries on the CHILD ■■■■■■■■ */
.card {
display: flex;
flex-direction: column; /* Default: stacked (narrow container) */
gap: 1rem;
}
```

```
/* When the CONTAINER (not viewport) is at least 400px wide */
@container (min-width: 400px) {
.card {
flex-direction: row; /* Side by side in wider containers */
}
.card__image {
width: 200px;
flex-shrink: 0;
}
}

@container (min-width: 600px) {
.card__title { font-size: 1.5rem; }
}

/* ■■■ Query a named container ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@container sidebar (min-width: 280px) {
.nav-item { flex-direction: row; align-items: center; }
}

/* ■■■ Container Query Units ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
/* cqi = 1% of container inline (width) size */
.card__title {
font-size: clamp(1rem, 4cqi, 1.5rem);
/* Font scales with CONTAINER width, not viewport */
}
```

| | Media Queries | Container Queries |
|---|---|---|
| Respond to | Viewport (whole screen) size | Parent container size |
| Component reusable? | No — coupled to viewport breakpoints | Yes — fully self-contained |
| When to use | Page-level layout, dark mode, print | Cards, widgets, sidebars, UI components |
| Setup required | None | container-type on parent element |
| Browser support | Universal | Chrome 105+, Firefox 110+, Safari 16+ |

**Q: What is the difference between a container query and a media query?**

A: Media queries respond to the viewport — the entire browser window size. Container queries respond to the size of a specific parent element. A card in a narrow sidebar and a wide main area needs different layouts — a media query cannot tell you the card's available space, only the viewport. A container query directly knows its parent's width and adapts accordingly. This makes components truly self-contained and reusable anywhere in the layout.

**Q: Why must container-type be on the PARENT, not the component itself?**

A: A component cannot query its own size — that would create a circular dependency: the size depends on the style, which depends on the size. You define containment on the parent, which tells the browser: track my size and make it available for queries from my children. The component queries this context and responds to its container's size, never its own — breaking the circularity.